TCP-Lab 讲义2：RDT-2.0

本讲义对应 **Step 2：RDT-2.0（解决位错/出错问题）**，在 RDT-1.0 的基础上加入**校验和机制**，让系统能够检测并恢复"数据在传输过程中被损坏"的情况。

---

1. RDT-2.0 是什么?

RDT-2.0 = 在底层信道可能出错的情况下实现可靠数据传输。

与 RDT-1.0 的区别：

- **RDT-1.0**：假设信道完全可靠（`eFlag=0`），不会出错、丢包或延迟
- **RDT-2.0**：允许信道出现**位错**（`eFlag=1`），但仍**不处理丢包和延迟**

1.1 什么是"位错"?

位错（bit error）是指数据在传输过程中，部分比特被翻转导致内容损坏。类比生活：

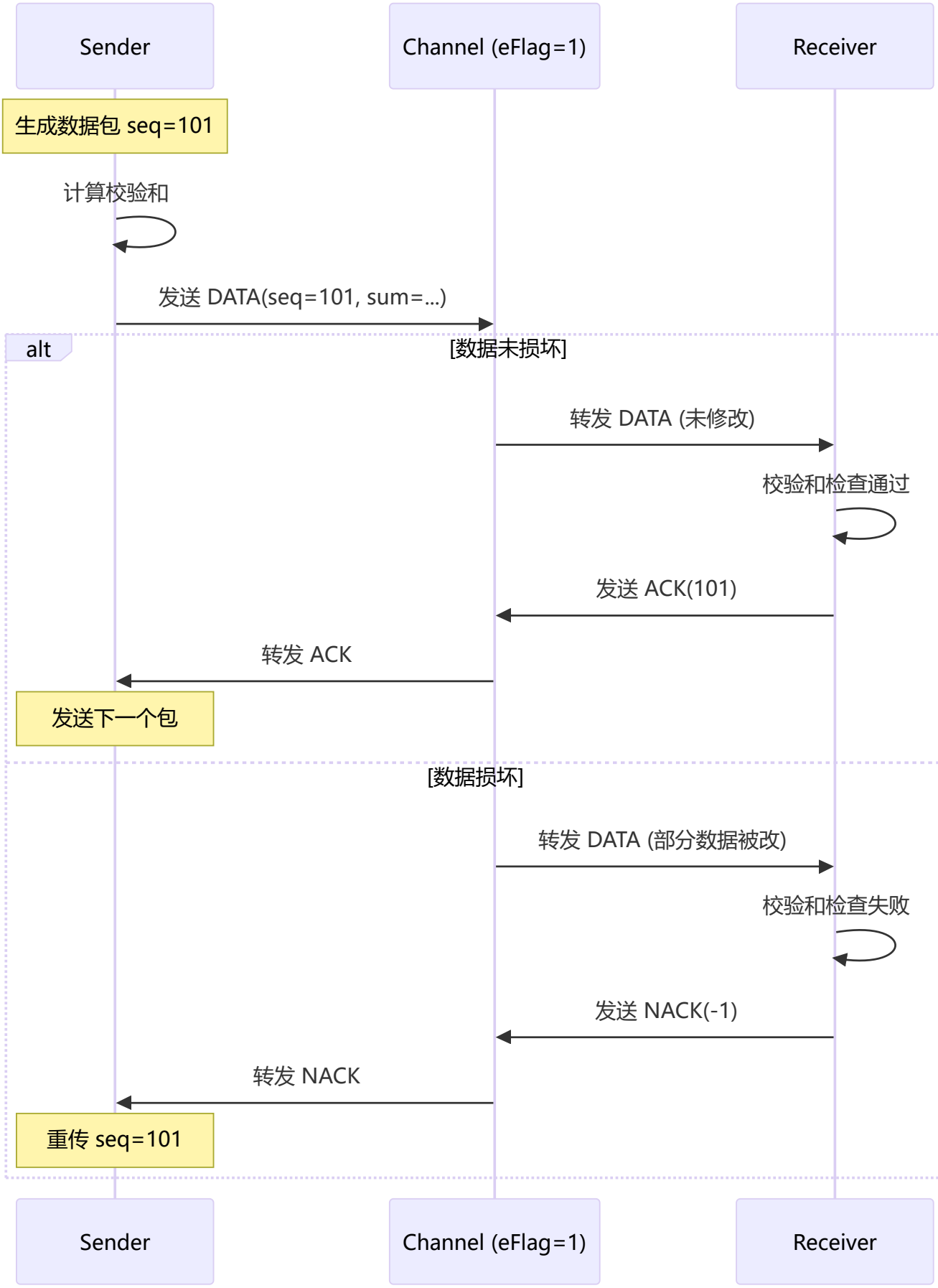
- 快递盒子到了，但盒子被压坏，里面的东西乱了
- 收件人能看到盒子，但内容不对

在本实验中，当 `eFlag=1` 时，模拟信道（Server）会以约 1% 的概率将数据段中的部分整数改成负数，或将 ACK 中的 `ack` 改成负数。

1.2 RDT-2.0 如何应对位错?

核心机制：**校验和（CheckSum） + 确认/否认（ACK/NACK）**

1. **发送端**：
    - 计算数据报的校验和
    - 将校验和放入 TCP 首部的 `th_sum` 字段
    - 发送数据报
    - 等待 ACK/NACK
2. **接收端**：
    - 收到数据报后重新计算校验和
    - 比对计算值与接收值：
        - **相等**：数据未损坏 → 发送 ACK（确认号 = seq）
        - **不等**：数据已损坏 → 发送 NACK（确认号 = -1）
3. **发送端收到反馈**：
    - 收到正确的 ACK（且确认号匹配）→ 发送下一个分组
    - 收到 NACK 或损坏的 ACK → 重传当前分组


Sender
Channel (eFlag=1)
Receiver
生成数据包 seq=101
计算校验和
发送 DATA(seq=101, sum=...)
alt
[数据未损坏]
转发 DATA (未修改)
校验和检查通过
发送 ACK(101)
转发 ACK
发送下一个包
[数据损坏]
转发 DATA (部分数据被改)
校验和检查失败
发送 NACK(-1)
转发 NACK
重传 seq=101
Sender
Channel (eFlag=1)
Receiver

2. 为什么需要校验和?

校验和是一种**错误检测码**，用于检测数据在传输过程中是否发生改变。

2.1 校验和的工作原理（16位反码求和）

本实验使用的是**16位反码求和算法**：

1. **发送端计算校验和**：

   ```
   将数据分成16位的段
   将所有段相加（超过16位的进位加回低位）
   对结果取反码
   ```

2. **接收端验证**：

   ```
   重新计算校验和
   与接收到的校验和比较
   相等 → 数据可能正确（概率上）
   不等 → 数据一定错误
   ```

2.2 校验和能检测所有错误吗?

不能。校验和只能检测大部分错误，但存在"概率碰撞"：

- 如果多个位错刚好使校验和保持不变，就检测不出来
- 但在实际应用中，这种概率非常低

更强的错误检测方法包括：CRC（循环冗余校验）、MD5、SHA 等。

---

3. RDT-2.0 实现详解

3.1 校验和计算（CheckSum.java）

需要校验的字段（按实验要求）：

- TCP 首部的 `seq`（4字节）
- TCP 首部的 `ack`（4字节）
- TCP 首部的 `sum`（2字节，**计算时视为0**）
- TCP 数据段 `data[]`（每个 int 为 4字节）

实现思路

```
public static short computeChkSum(TCP_PACKET tcpPack) {
    int sum = 0;
    TCP_HEADER header = tcpPack.getTcpH();

    // 1. 校验seq字段（分成两个16位）
    int seq = header.getTh_seq();
```

```
        sum += (seq >> 16) & 0xFFFF;  // 高16位
        sum += seq & 0xFFFF;           // 低16位

        // 2. 校验ack字段
        int ack = header.getTh_ack();
        sum += (ack >> 16) & 0xFFFF;
        sum += ack & 0xFFFF;

        // 3. sum字段计算时视为0，不参与累加

        // 4. 校验数据段
        int[] data = tcpPack.getTcpS().getData();
        if (data != null) {
            for (int i = 0; i < data.length; i++) {
                int value = data[i];
                sum += (value >> 16) & 0xFFFF;
                sum += value & 0xFFFF;
            }
        }

        // 处理进位
        while ((sum >> 16) != 0) {
            sum = (sum & 0xFFFF) + (sum >> 16);
        }

        // 取反得到校验和
        sum = ~sum;

        return (short) (sum & 0xFFFF);
    }
```

关键注意点

1. **Java 没有无符号类型**：
   - 使用 `& 0xFFFF` 确保只取低 16 位
   - 使用 `& 0xFF` 确保只取低 8 位
2. **计算时 sum 字段为 0**：
   - 发送前先计算校验和（此时 `th_sum` 默认为 0）
   - 计算结果写入 `th_sum`
   - 接收端重新计算时，会用接收到的 `th_sum` 参与计算（但在我们的实现中，我们直接跳过 sum 字段不参与计算）
3. **进位处理**：
   - 每次加法可能产生超过 16 位的结果
   - 需要将高位加回低位，直到不再有进位

3.2 发送端修改（TCP_Sender.java）

修改点 1：设置 eFlag

```
public void udt_send(TCP_PACKET stcpPack) {
    // RDT-2.0：使用eFlag=1（只可能出错）
    tcpH.setTh_eflag((byte)1);
    client.send(stcpPack);
}
```

修改点 2：处理接收到的 ACK

```
public void recv(TCP_PACKET recvPack) {
    // 检查ACK的校验和
    if (CheckSum.computeChkSum(recvPack) == recvPack.getTcpH().getTh_sum()) {
        // 校验和正确
        System.out.println("Receive ACK Number: " + recvPack.getTcpH().getTh_ack());
        ackQueue.add(recvPack.getTcpH().getTh_ack());
    } else {
        // 校验和错误，ACK损坏，视为NACK
        System.out.println("Receive Corrupted ACK, treat as NACK");
        ackQueue.add(-1);
    }

    waitACK();
}
```

修改点 3：waitACK 逻辑（已有，无需修改）

```
public void waitACK() {
    if (!ackQueue.isEmpty()) {
        int currentAck = ackQueue.poll();
        if (currentAck == tcpPack.getTcpH().getTh_seq()) {
            // 收到正确ACK，设置flag=1，让rdt_send继续
            flag = 1;
        } else {
            // 收到NACK或错误ACK，重传
            System.out.println("Retransmit: " + tcpPack.getTcpH().getTh_seq());
            udt_send(tcpPack);
            flag = 0;
        }
    }
}
```

3.3 接收端修改（TCP_Receiver.java）

修改点 1：rdt_recv 中的校验

```
public void rdt_recv(TCP_PACKET recvPack) {
    // 检查校验和
    if (CheckSum.computeChkSum(recvPack) == recvPack.getTcpH().getTh_sum()) {
        // 校验通过，数据正确
        System.out.println("Receive correct packet, seq: " +
recvPack.getTcpH().getTh_seq());

        // 生成ACK
        tcpH.setTh_ack(recvPack.getTcpH().getTh_seq());
        ackPack = new TCP_PACKET(tcpH, tcpS, recvPack.getSourceAddr());
        tcpH.setTh_sum(CheckSum.computeChkSum(ackPack));
        reply(ackPack);

        // 交付数据
        dataQueue.add(recvPack.getTcpS().getData());
        sequence++;
    } else {
        // 校验失败，数据损坏
        System.out.println("Receive corrupted packet!");
        System.out.println("  Packet seq: " + recvPack.getTcpH().getTh_seq());

        // 发送NACK（确认号为-1）
        tcpH.setTh_ack(-1);
        ackPack = new TCP_PACKET(tcpH, tcpS, recvPack.getSourceAddr());
        tcpH.setTh_sum(CheckSum.computeChkSum(ackPack));
        reply(ackPack);
    }

    // 每20组交付一次
    if (dataQueue.size() == 20)
        deliver_data();
}
```

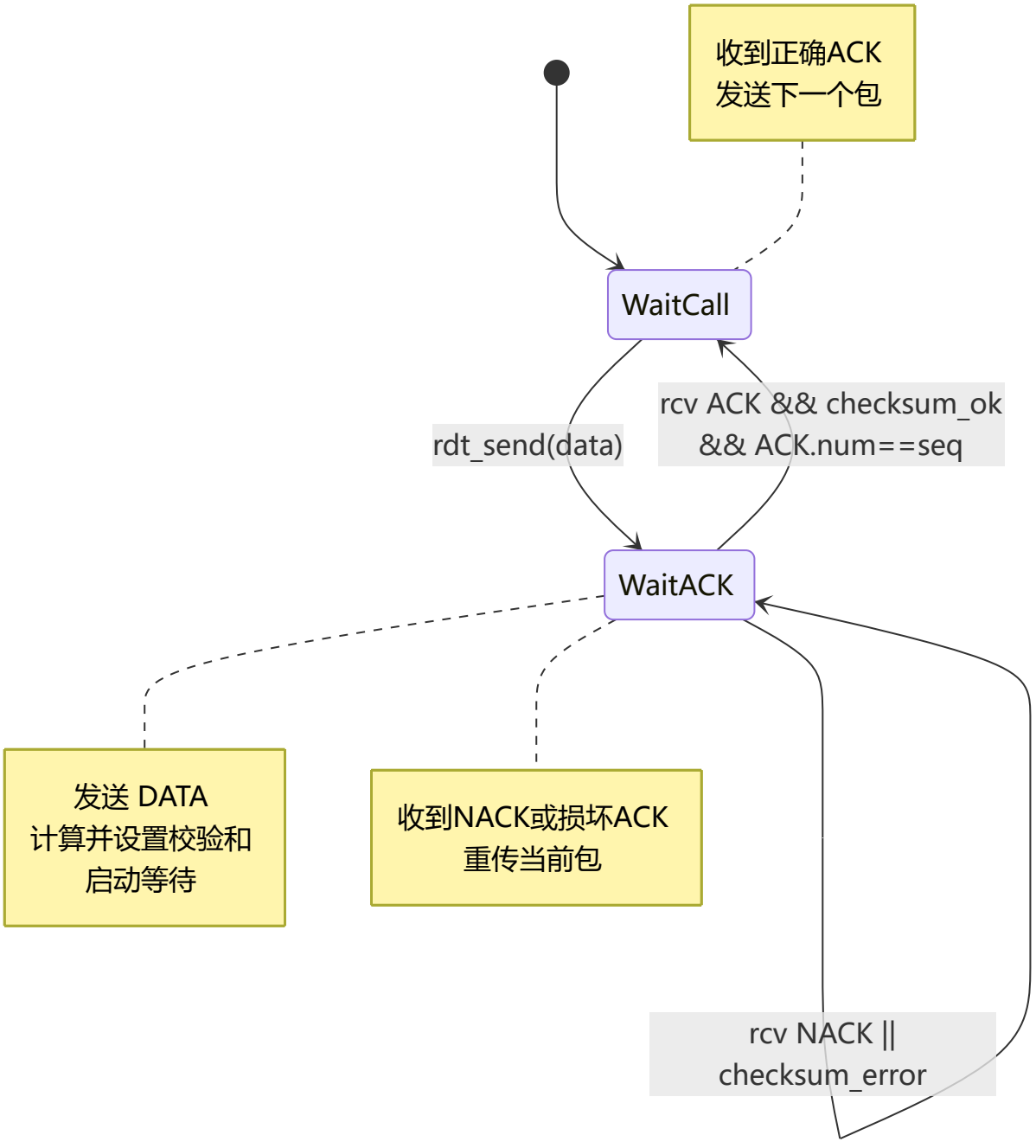
修改点 2：reply 中设置 eFlag

```
public void reply(TCP_PACKET replyPack) {
    // RDT-2.0：ACK也可能出错
    tcpH.setTh_eflag((byte)1);
    client.send(replyPack);
}
```

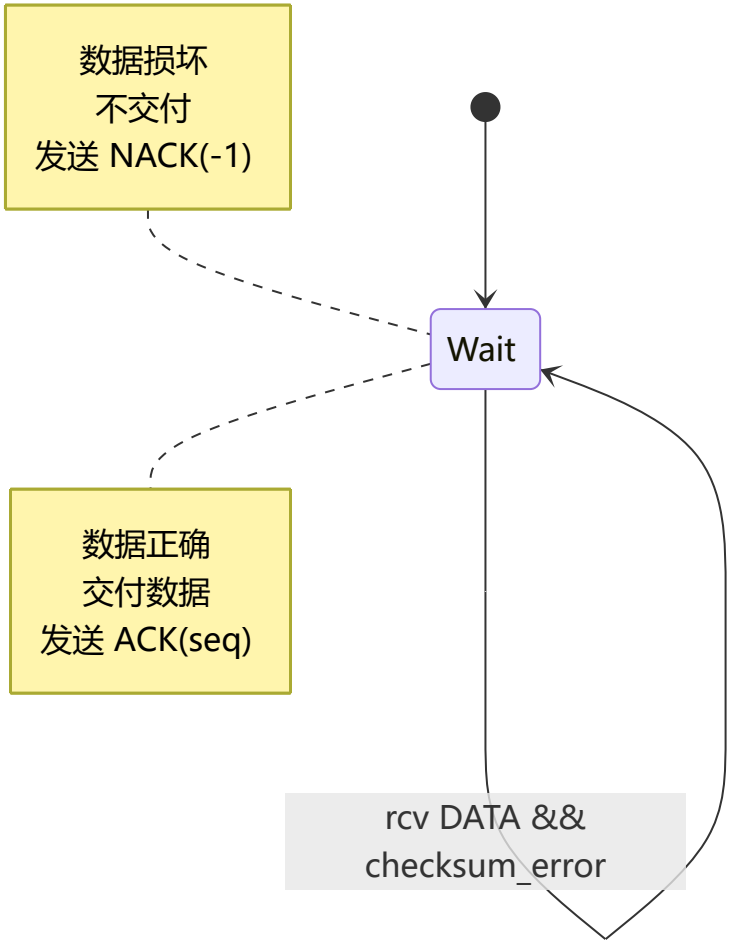
---

4. RDT-2.0 的状态机模型

4.1 发送端状态机



4.2 接收端状态机



---

5. 运行与验证

5.1 验证步骤

1. **编译运行**：

```
mvn clean package
java -jar target/TCP_Test_final-1.0-SNAPSHOT.jar
```

2. **观察日志**：
   - 正常情况：`Receive correct packet, seq: 101/201/301...`
   - 出错情况：`Receive corrupted packet!` → `Retransmit: 101`
   - ACK 损坏：`Receive Corrupted ACK, treat as NACK` → `Retransmit: 101`
3. **检查输出文件**：
   - recvData.txt：应包含完整数据，无损坏值，无重复
   - Log.txt：可用可视化工具分析重传情况

5.2 预期现象（eFlag=1）

在 `eFlag=1`（只出错）模式下：

- ✅ 大部分包能正常传输
- ✅ 约 1% 的包会出错并触发重传
- ✅ `recvData.txt` 最终包含完整正确的数据
- ✅ 不应出现负数或明显错误的数值
- ✅ 不应出现重复交付（行数应正确）

5.3 常见问题排查

问题 1：校验和总是不匹配

可能原因：

- 发送端和接收端的校验和计算方法不一致
- 忘记在计算前将 `th_sum` 字段设为 0
- 字节序或位运算错误

调试方法：

```
// 在发送端打印
System.out.println("Send checksum: " + tcpH.getTh_sum());

// 在接收端打印
System.out.println("Recv checksum: " + recvPack.getTcpH().getTh_sum());
System.out.println("Calc checksum: " + CheckSum.computeChkSum(recvPack));
```

问题 2：收到 NACK 后没有重传

可能原因：

- `waitACK()` 中的重传逻辑有误
- `flag` 标志位没有正确设置

检查点：

- 确保 `currentAck != tcpPack.seq` 时调用 `udt_send(tcpPack)`
- 确保 `flag` 保持为 0，让 `rdt_send` 继续等待

问题 3：数据重复交付

可能原因：

- 收到重复包时也执行了 `dataQueue.add()`
- RDT-2.0 本身不处理重复，这是 RDT-2.2 的内容

说明：

- RDT-2.0 在 `eFlag=1` 下理论上不会产生重复包

- 如果出现重复，可能是网络延迟导致（这属于 RDT-3.0 要解决的问题）

---

6. RDT-2.0 的局限性

虽然 RDT-2.0 能处理位错，但仍有三个明显局限：

6.1 ACK/NACK 本身可能损坏

问题：如果 ACK 在传输中损坏，发送端无法判断到底是"数据正确但 ACK 损坏"还是"数据损坏"。

解决：

- 当前实现中，我们对 ACK 也进行校验和检查
- 如果 ACK 损坏，将其视为 NACK，触发重传
- 但这可能导致不必要的重传（数据其实正确到达了）

6.2 不能处理丢包

问题：如果数据包或 ACK 在网络中丢失（根本没到达），发送端会无限等待。

解决：RDT-3.0 引入**超时重传机制**

6.3 不能处理重复包

问题：当 ACK 损坏导致发送端重传时，接收端可能会收到重复的包。

解决：RDT-2.2 引入**序号机制**，让接收端能识别并丢弃重复包

---

7. 从 RDT-2.0 到 RDT-2.2（预告）

RDT-2.2 的主要改进：

1. **取消 NACK**：只使用 ACK
2. **引入序号**：区分"新包"和"重复包"
3. **重复 ACK 等价于 NACK**：如果发送端连续收到相同的 ACK，说明新包没被正确接收

这样可以简化协议，并为后续的滑动窗口打基础。

---

8. 总结

8.1 RDT-2.0 核心要点

方面	内容
目标	在有位错的信道上实现可靠传输
机制	校验和 + ACK/NACK
校验字段	seq, ack, sum(视为0), data[]

方面	内容
eFlag 设置	1（只可能出错）
发送端	计算校验和，等待 ACK，收到 NACK 则重传
接收端	验证校验和，正确则发 ACK，错误则发 NACK
局限性	不处理丢包、延迟、重复包

8.2 实现检查清单

- [x] 实现 `CheckSum.computeChkSum()`（16位反码求和）
- [x] 发送端在 `rdt_send` 中计算并设置校验和
- [x] 发送端设置 `eFlag=1`
- [x] 接收端在 `rdt_recv` 中验证校验和
- [x] 接收端根据校验结果发送 ACK 或 NACK
- [x] 接收端设置 `eFlag=1`
- [x] 发送端在 `recv` 中检查 ACK 的校验和
- [x] 发送端在 `waitACK` 中根据 ACK/NACK 决定重传或前进
- [x] 测试 `eFlag=1` 下能正确检测错误并重传
- [x] 验证 `recvData.txt` 无错误数据

8.3 下一步

继续学习 **RDT-2.2**（取消 NACK，使用序号）。

附录：关键代码对照表

文件	函数	RDT-1.0	RDT-2.0 变化
CheckSum.java	computeChkSum	返回 0	**实现16位反码求和**
TCP_Sender.java	udt_send	eFlag=0	**eFlag=1**
TCP_Sender.java	recv	直接加入队列	**检查ACK校验和**
TCP_Receiver.java	rdt_recv	假设数据正确	**检查数据校验和，发ACK/NACK**
TCP_Receiver.java	reply	eFlag=0	**eFlag=1**
